SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Computer Vision with OpenCV COURSE CODE: DSA0202

COURSE OUTCOME COVERED

CO5: *Apply computer vision techniques to real-world applications such as face recognition, tracking, medical imaging, and autonomous systems. (BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 5 Total Marks:50 Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I CH.V.Ramayogi/192424319(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: CH.V.Ramayogi

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	

Annexure A

Constraint-Based Problem Solving

1. Scenario: A company wants to deploy a secure access system using face recognition in a low-light environment.

Constraints:

A. Limited computational resources (embedded device)

B. Faces may appear at different orientations (±30°)

C. Real-time recognition required (<200 ms per face)

Questions:

1. How would you adapt Eigenface-based recognition to handle varying illumination and orientation?
2. Suggest preprocessing steps to improve recognition under low-light conditions.
3. Discuss strategies to optimize computation for real-time performance on constrained hardware.

Question 2: Photo Album Organization

2. Scenario: You are designing an AI system to automatically organize a large personal photo collection by people and events.

Constraints:

A. Millions of images with inconsistent resolution and quality

B. Multiple photos per person in different poses

C. Privacy considerations: all processing must occur locally

Questions:

1. How would you use face recognition and clustering to group photos of the same individual?
2. Propose a method to detect and separate images with occlusions or partial faces.
3. Discuss trade-offs between accuracy and computational efficiency for local processing.

3. Scenario: A smart city surveillance system must detect moving objects and track pedestrians

across multiple cameras.

Constraints:

A. Crowded scenes with frequent occlusion

B. Limited network bandwidth between cameras

C. Real-time processing requirement

Questions:

1. How would you design a foreground–background separation algorithm to minimize false positives in crowded scenes?
2. Explain how particle filters can be used to handle occlusion during tracking. 3. Propose an efficient multi-camera strategy for consistent pedestrian identification across viewpoints.

4. Scenario: A self-driving car must navigate urban streets while detecting lanes, road signs, and pedestrians.

Constraints:

A. Variable weather and lighting conditions (rain, night)

B. Real-time decision-making (≤50 ms per frame)

C. High-density traffic with moving pedestrians

Questions:

1. How would you implement road detection robust to shadows, rain, and nighttime illumination?
2. Propose a pipeline to simultaneously detect road signs and pedestrians efficiently.
3. Discuss the trade-offs between deep learning-based detectors and classical computer vision methods under real-time constraints.

5. Scenario: An AR system for surgical assistance must overlay segmented organs in real time.

Constraints:

A. High-resolution 3D medical images (CT/MRI)

B. Latency must be <100 ms for surgeon interaction

C. Limited GPU memory on wearable AR headset

Questions:

1. How would you implement real-time organ segmentation under these memory and latency constraints?
2. Discuss methods to integrate segmentation results accurately in AR/MR overlays.
3. Suggest strategies to reduce computational load while maintaining segmentation accuracy.

Answers:

1. Secure Face Recognition in Low-Light Environment

1. How would you adapt Eigenface-based recognition to handle varying illumination and orientation?

Eigenfaces are sensitive to changes in lighting and face pose. To improve them:

- Apply illumination normalization before extracting Eigenfaces.
- Use histogram equalization or CLAHE to reduce lighting differences.
- Align the detected face using the eyes/nose landmarks.
- Generate or use multiple face templates for different orientations such as −30°, 0°, and +30°.
- Use PCA separately on normalized and aligned face images.
- For larger pose variations, combine Eigenfaces with a pose-normalization technique.

This makes the system more robust to illumination and moderate head rotation.

2. Preprocessing steps for low-light conditions

The following preprocessing pipeline can be used:

1. **Face detection** – locate the face in the image.
2. **Face alignment** – align the eyes and other landmarks.
3. **Noise removal** – use a lightweight Gaussian or median filter.
4. **Brightness enhancement** – use gamma correction.
5. **Contrast enhancement** – use CLAHE.
6. **Illumination normalization** – reduce shadows and uneven lighting.
7. **Grayscale conversion** – required for traditional Eigenface processing.
8. **Image resizing** – resize to a small fixed size such as 64×64 or 100×100.

3. Strategies to optimize computation for real-time performance

Since the device has limited computational resources:

- Use small face images instead of high-resolution images.
- Reduce the number of Eigenfaces using PCA.
- Perform face detection only when necessary.
- Use region-of-interest (ROI) processing.
- Store precomputed Eigenface projections of registered users.
- Use efficient distance calculations such as Euclidean distance.
- Use integer/fixed-point operations where suitable.
- Use lightweight face detection models.
- Process only the face region rather than the complete frame.

The goal is to keep the complete recognition pipeline below 200 ms per face.

2. Photo Album Organization

1. How would you use face recognition and clustering to group photos of the same individual?

A suitable pipeline is:

Image → Face Detection → Face Alignment → Face Feature Extraction → Clustering → Person Groups

Steps:

1. Detect all faces in every image.
2. Align each detected face.
3. Extract a compact face embedding/feature vector.
4. Compare feature vectors between faces.
5. Apply clustering such as DBSCAN or hierarchical clustering.
6. Faces with similar feature vectors are grouped together.
7. Assign a person label after confirming representative images.

For millions of images, approximate nearest-neighbor search can be used to make matching faster.

2. Method to detect and separate occluded or partial faces

Use:

- Face detection confidence score to identify unreliable detections.
- Detect facial landmarks such as eyes, nose and mouth.
- Check whether important landmarks are missing.

- Calculate the visible face area.
- Use an occlusion classifier to classify faces as fully visible or partially occluded.
- Store partial faces separately instead of allowing them to create incorrect clusters.

For example:

Full face → Normal clustering

Partial/occluded face → Occlusion group → Match only when confidence is high

3. Trade-offs between accuracy and computational efficiency

Approach	Accuracy	Computation
Traditional face features	Medium	Low
Lightweight embeddings	High	Medium
Large deeplearning model	Very high	High
High-resolution processing	High	High
Low-resolution processing	Moderate	Low

For local processing, a lightweight face-embedding model + efficient clustering is a good compromise.

Privacy is also improved because the images and face features never need to leave the user's device.

3. Smart City Surveillance and Multi-Camera Tracking

1. Foreground-background separation to minimize false positives

A robust approach can combine background subtraction with object detection.

Pipeline:

Video → Background Model → Foreground Mask → Noise Removal → Object Detection → Tracking

To reduce false positives:

- Maintain an adaptive background model.
- Use temporal information from multiple frames.
- Apply morphological operations such as erosion and dilation.
- Remove very small foreground regions.
- Use pedestrian/object detection to verify foreground objects.
- Use shadow detection to avoid detecting shadows as pedestrians.
- Update the background gradually rather than immediately.

In crowded scenes, combining motion information with object detection is more reliable than simple frame differencing.

2. How particle filters handle occlusion during tracking

A particle filter represents the possible position of a pedestrian using many particles.

For example:

Current position → Predict possible positions → Compare with observation → Update particle weights → Estimate position

During normal tracking:

- Particles are concentrated around the detected pedestrian.
- The filter predicts the next position.

During occlusion:

- The pedestrian may disappear temporarily.
- The particle filter continues predicting the pedestrian's likely location using previous position and motion.
- When the person becomes visible again, new observations update the particle weights.
- The tracker can then recover the correct trajectory.

Therefore, particle filters are useful when observations are temporarily missing or noisy.

3. Efficient multi-camera strategy

Use a distributed tracking architecture:

1. Each camera performs local object detection.
2. Each camera extracts a compact pedestrian feature/embedding.
3. Only metadata and feature vectors are transmitted instead of complete video.
4. A central or edge server performs cross-camera matching.
5. Use appearance features, time, camera location and movement direction.

6. Assign a consistent temporary identity across cameras.

This reduces network bandwidth because raw video does not need to be continuously transmitted.

4. Self-Driving Car

1. Road detection robust to shadows, rain and nighttime illumination

A robust road-detection pipeline can use:

Camera → Image Enhancement → ROI Selection → Road Segmentation → Lane Detection

Methods:

- Use HDR or exposure adjustment for bright/dark conditions.
- Apply gamma correction for nighttime images.
- Use contrast enhancement.
- Use color spaces such as HSV or HLS instead of only RGB.
- Detect and remove shadow regions.
- Use temporal information from consecutive frames.
- Use image de-raining techniques when necessary.
- Use lane-marking detection using edge detection and Hough transform.
- For difficult conditions, use a lightweight deep-learning segmentation model.

Combining multiple cues makes road detection more reliable under different weather and lighting conditions.

2. Pipeline to detect road signs and pedestrians efficiently

A suitable pipeline is:

Camera Input

↓

Image Preprocessing

↓

ROI Selection

↓

Lightweight Object Detector

↙ ↘

Road Signs Pedestrians

↓ ↓

Classification Tracking

↓ ↓

Decision/Control System

To improve efficiency:

- Process only relevant regions.
- Use a lightweight detector.
- Use lower resolution for distant objects.
- Track already detected objects between frames instead of detecting everything from scratch.
- Use hardware acceleration when available.

3. Deep learning vs classical computer vision

Feature	**Deep Learning**	**Classical Computer Vision**
Accuracy	High	Moderate
Complex environments	Better	Limited
Lighting robustness	Better after training	Lower
Computation	High	Low
Training required	Yes	Usually no
Hardware requirement	Higher	Lower
Real-time on powerful hardware	Good	Excellent
Adaptability	High	Lower

For a self-driving car, a **hybrid approach** is useful. Lightweight deep-learning detectors can handle pedestrians and signs, while classical techniques can perform fast lane and edge processing.

The main challenge is meeting the **≤50 ms/frame** requirement, so model size, input resolution and hardware acceleration must be carefully optimized.

5. AR Surgical Assistance – Real-Time Organ Segmentation

1. Real-time organ segmentation under memory and latency constraints

A lightweight segmentation pipeline can be used:

CT/MRI Data → Preprocessing → Lightweight Segmentation Network → Organ Mask → 3D Reconstruction → AR Overlay

To meet the <100 ms latency requirement:

- Use a lightweight CNN such as a compact U-Net variant.
- Reduce input resolution where possible.
- Process only the relevant region of interest (ROI).
- Use 2D or 2.5D processing instead of full 3D processing when appropriate.
- Use model quantization.
- Use pruning to remove unnecessary network parameters.
- Use reduced-precision computation such as FP16/INT8 when supported.
- Reuse previous segmentation results instead of recomputing everything.

2. Integrating segmentation accurately into AR/MR overlays

The segmentation mask must be correctly aligned with the real surgical environment.

Steps:

1. Segment the target organ.
2. Convert the segmentation mask into a 3D representation.
3. Register the medical image coordinates with the patient's physical coordinates.
4. Use camera tracking and spatial registration.
5. Align the 3D organ model with the patient's anatomy.
6. Continuously update the overlay when the patient or camera moves.
7. Use depth information to maintain correct positioning.

Accurate image-to-patient registration is critical because even a small alignment error can make the overlay misleading.

3. Strategies to reduce computational load while maintaining accuracy

Important optimization techniques include:

- **Model pruning** – remove unnecessary network parameters.
- **Quantization** – use INT8/FP16 instead of full precision.
- **Knowledge distillation** – train a small model using a larger model as a teacher.
- **ROI processing** – process only the required anatomical region.
- **Lower-resolution input** where clinically acceptable.

- **Efficient network architectures** such as lightweight U-Net variants.
- **Temporal reuse** – update segmentation only when significant changes occur.
- **GPU/CPU optimization** – use hardware acceleration available on the headset.
- **Caching** – reuse previously calculated features.

The best solution is a lightweight segmentation model + ROI processing + quantization + temporal reuse, which reduces memory and computation while maintaining acceptable segmentation accuracy.